CURSO 2026 - 2027

MANUAL DE REFERENCIA DEL ALUMNADO

UT01 · Fundamentos de Bases de Datos NoSQL y MongoDB

CICLO FORMATIVO

Desarrollo de Aplicaciones Web (DAW) · 2º curso

MÓDULO

Tecnologías Web Actuales (Optativa)

UNIDAD DE TRABAJO

UT01 · Fundamentos MongoDB



Miguel Ángel Rubiano Barros
I.E.S. Suárez de Figueroa — Departamento de Informática y Comunicaciones

Índice de contenidos

Capítulo 1 · Introducción a las bases de datos NoSQL 3

Capítulo 2 · Introducción a MongoDB 6

Capítulo 3 · Instalación y herramientas de trabajo 9

Capítulo 4 · Tipos de datos en MongoDB (BSON) 12

Capítulo 5 · Bases de datos y colecciones 15

Capítulo 6 · Operaciones CRUD 17

Capítulo 7 · Índices y rendimiento de consultas 25

Capítulo 8 · Exportación e importación de datos 32

Capítulo 9 · Copias de seguridad y restauración 35

Capítulo 10 · Actividad 2: práctica integral de CRUD 38

Capítulo 11 · Cierre de la unidad y próximos pasos 43

Capítulo 1 · Introducción a las bases de datos NoSQL

Por qué han surgido las bases de datos NoSQL, qué problemas resuelven y en qué se diferencian de las bases de datos relacionales que ya conoces.

1.1 ¿Qué son las bases de datos NoSQL?

El término **NoSQL** no significa "sin SQL", sino **"Not Only SQL"** (no solo SQL). Se utiliza para referirse a una amplia familia de sistemas de gestión de bases de datos que **no siguen el modelo relacional tradicional** (el de tablas, filas y columnas con un esquema fijo que ya conoces de asignaturas anteriores).

En lugar de forzar los datos a encajar en tablas rígidas, las bases de datos NoSQL se diseñaron pensando en:

- **Manejar grandes volúmenes de datos**, a menudo no estructurados o semiestructurados (texto libre, documentos JSON, registros de sensores, contenido multimedia, etc.).
- **Escalar de forma horizontal**: en lugar de comprar un servidor cada vez más potente (escalado vertical), se añaden más servidores que se reparten el trabajo.
- **Ofrecer un esquema flexible**, de modo que no todos los registros tienen por qué tener exactamente los mismos campos.
- **Optimizarse para operaciones concretas**: hay motores NoSQL pensados para lecturas ultrarrápidas, otros para relaciones complejas, otros para series temporales, etc.

Estas bases de datos surgen como respuesta a varios retos que las bases de datos relacionales clásicas no resolvían bien: la explosión del volumen de datos generados en internet, las limitaciones del modelo relacional para ciertos casos de uso (datos muy variables, jerárquicos o muy interconectados) y la necesidad de una escalabilidad que el modelo relacional tradicional no ofrecía de forma sencilla.

> **Idea clave.** La idea central que debes retener de este apartado es que **NoSQL no es una tecnología concreta, sino una categoría** que agrupa enfoques muy distintos entre sí. MongoDB, que estudiaremos en esta unidad, es solo uno de los representantes de esa categoría, concretamente de la familia *documental*.

1.2 Tipos de bases de datos NoSQL

Dentro de NoSQL existen varios modelos de datos muy distintos entre sí. Conviene distinguirlos porque cada uno resuelve un problema diferente y, en un proyecto real, es habitual combinar varios tipos de bases de datos según la necesidad (lo que se conoce como *polyglot persistence*).

Bases de datos documentales

Almacenan la información en **colecciones de documentos**, normalmente en formato JSON o similar, donde cada documento es un conjunto de pares clave-valor que puede anidar otros documentos y listas. Es el modelo que usa **MongoDB** (y también Firebase). Cada documento puede tener una

estructura ligeramente distinta a los demás documentos de su misma colección, lo que da mucha flexibilidad.

Bases de datos clave-valor

El modelo más simple: cada dato se identifica mediante una clave única y se recupera por esa clave, de forma parecida a un diccionario o a un mapa en programación. Permiten velocidades de lectura y escritura muy altas y se usan sobre todo en sistemas de **caché** (por ejemplo, para no repetir consultas costosas) o en almacenamiento de sesiones de usuario. Ejemplos: **DynamoDB, Redis, Riak**.

Bases de datos columnares

Organizan los datos en tablas, pero almacenándolos físicamente por columnas en lugar de por filas, lo que permite que cada fila tenga columnas ligeramente distintas y optimiza mucho las consultas que leen pocas columnas de muchísimas filas (típico en analítica de grandes volúmenes). Ejemplos: **Cassandra, Bigtable**.

Bases de datos de grafos

Están pensadas para datos muy interconectados: almacenan **nodos** (entidades) y **aristas** (relaciones entre ellas), y permiten recorrer esas relaciones de forma muy eficiente. Son ideales para redes sociales, sistemas de recomendación o detección de fraude, donde lo importante no son tanto los datos en sí como las conexiones entre ellos. Ejemplos: **Neo4j, OrientDB**.

Modelo	Unidad de almacenamiento	Ejemplos	Caso de uso típico
`Documental`	Documento JSON/BSON	MongoDB, Firebase	Catálogos, contenido web, APIs
`Clave-valor`	Par clave → valor	Redis, DynamoDB, Riak	Caché, sesiones, colas
`Columnar`	Columnas de una tabla ancha	Cassandra, Bigtable	Analítica sobre grandes volúmenes
`Grafos`	Nodos y aristas	Neo4j, OrientDB	Redes sociales, recomendaciones

1.3 SQL vs NoSQL: comparativa

Antes de seguir avanzando merece la pena fijar con claridad en qué se parecen y en qué se diferencian ambos mundos, porque a lo largo del curso compararemos constantemente las operaciones de MongoDB con el SQL que ya conoces.

Bases de datos SQL	Bases de datos NoSQL
`Esquema fijo y predefinido`	Esquema flexible y dinámico
`Modelo relacional (tablas)`	Modelos variados (documentos, grafos, clave-valor, columnas…)
`Escalabilidad vertical (servidor más potente)`	Escalabilidad horizontal (más servidores)

Bases de datos SQL	Bases de datos NoSQL
Propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad)	Modelo BASE (Basically Available, Soft state, Eventual consistency)
Lenguaje SQL estandarizado	APIs específicas de cada base de datos
Ideal para datos estructurados y relaciones complejas	Ideal para datos semiestructurados o no estructurados

Para saber más. MongoDB, desde sus versiones más recientes, también ofrece **transacciones multi-documento** con garantías ACID (lo veremos más adelante, en el bloque de MongoDB avanzado). La frontera entre SQL y NoSQL es hoy más difusa de lo que lo era hace unos años: conviene entender las comparativas como "tendencias generales", no como reglas absolutas.

1.4 Ventajas y casos de uso de NoSQL

Principales ventajas

- **Flexibilidad**: esquemas dinámicos que se adaptan fácilmente a cambios en los requisitos, sin necesidad de migraciones complejas.
- **Escalabilidad**: distribución horizontal de los datos entre varios servidores, pensada para manejar grandes volúmenes y alta disponibilidad.
- **Rendimiento**: al estar optimizadas para operaciones específicas y tener menos sobrecarga por relaciones entre tablas, suelen ofrecer consultas más simples y rápidas en esos casos de uso concretos.
- **Variedad**: al existir distintos modelos, se puede elegir el más adecuado a cada problema en lugar de forzar todos los datos a encajar en tablas.

¿Cuándo tiene sentido elegir NoSQL (y en concreto MongoDB)?

Como veremos con más detalle en el capítulo siguiente, NoSQL no es una solución universal: es una herramienta más que hay que elegir según el problema. En general, conviene valorar una base de datos documental como MongoDB cuando los datos son variables o evolucionan con frecuencia, cuando se prioriza la velocidad de desarrollo y la escalabilidad frente a las relaciones complejas, o cuando el propio dominio del problema encaja de forma natural con documentos (un producto de una tienda, un perfil de usuario, un artículo de un blog…).

Capítulo 2 · Introducción a MongoDB

Qué es MongoDB, cómo organiza la información y qué herramientas forman su ecosistema.

2.1 ¿Qué es MongoDB?

MongoDB es una base de datos orientada a documentos, de código abierto y escrita en C++. Es, con diferencia, la base de datos NoSQL documental más utilizada en la actualidad, tanto en proyectos personales y académicos como en entornos profesionales de gran escala.

Característica	Valor
`Tipo`	Base de datos documental
`Lanzamiento`	2009
`Desarrollador`	MongoDB Inc.
`Lenguaje de implementación`	C++
`Formato de almacenamiento`	BSON (Binary JSON)
`Licencia`	Código abierto (Server Side Public License desde la v4.4)

Sus características principales son el almacenamiento de datos en **documentos BSON** (una representación binaria de JSON, más eficiente y con tipos de datos adicionales, que veremos en el Capítulo 4), un **esquema flexible** (dos documentos de la misma colección pueden tener campos distintos) y una **escalabilidad horizontal** nativa mediante particionado de datos (*sharding*).

2.2 Arquitectura de MongoDB: documentos, colecciones y bases de datos

MongoDB organiza la información en tres niveles, de menor a mayor:

- **Documento**: la unidad básica de almacenamiento de datos. Es el equivalente a una fila de una tabla SQL, pero en formato BSON, similar a un objeto JSON, y sin un esquema fijo obligatorio.
- **Colección**: un grupo de documentos, normalmente con una estructura o propósito similar. Es el equivalente aproximado a una tabla SQL, salvo que no impone (por defecto) que todos sus documentos compartan exactamente los mismos campos.
- **Base de datos**: un contenedor de colecciones. Un mismo servidor de MongoDB puede alojar múltiples bases de datos, cada una con su propio conjunto de colecciones y su propio *namespace* (espacio de nombres) único.

Esta jerarquía —**base de datos → colección → documento**— es la que usaremos durante todo el curso, y conviene tenerla muy clara porque aparece constantemente en la sintaxis de los comandos: por ejemplo, `db.productos.find()` se lee como "en la base de datos actual (`db`), dentro de la colección `productos`, busca documentos".

Ejemplo de documento MongoDB

Un documento de MongoDB se parece mucho a un objeto JSON. Puede contener valores simples, pero también **documentos anidados** (objetos dentro de objetos) y **arrays**, algo que una fila de una tabla SQL no puede hacer directamente:

```
{
  "nombre": "Juan Perez",
  "edad": 30,
  "email": "juan.perez@email.com",
  "direccion": {
    "calle": "Calle Principal 123",
    "ciudad": "Madrid",
    "codigo_postal": "28001"
  },
  "telefonos": [
    "+34 912 345 678",
    "+34 600 123 456"
  ]
}
```

Un único documento puede describir una entidad completa, incluidas sus relaciones "internas" (dirección) y listas de valores (teléfonos), sin necesidad de tablas adicionales.

Idea clave. Observa que el documento anterior resolvería, en una base de datos relacional, al menos **tres tablas distintas** (personas, direcciones y teléfonos) relacionadas mediante claves foráneas. En MongoDB, toda esa información puede vivir en un único documento autocontenido. Esta decisión de diseño —"empotrar" información relacionada dentro de un mismo documento— es una de las ideas más importantes del modelado de datos en MongoDB, y la retomaremos en profundidad en la Unidad de Trabajo 2.

2.3 Características técnicas de MongoDB

Consultas

- Operaciones **CRUD** completas (Create, Read, Update, Delete), que estudiaremos en detalle en el Capítulo 6.
- **Agregaciones**: un framework propio para transformar, filtrar y resumir datos en varias etapas encadenadas (se estudia en la Unidad de Trabajo 2).
- **Índices secundarios**: estructuras adicionales para acelerar las consultas sobre campos concretos (Capítulo 7).

Rendimiento

- Almacenamiento y gestión eficiente en memoria de los datos usados con más frecuencia.
- **Réplicas** (*replica sets*) que permiten servir lecturas desde varios servidores.
- **Sharding** automático para repartir una colección muy grande entre varios servidores.

Seguridad y disponibilidad

- Mecanismos de **autenticación** y **autorización** basados en usuarios y roles.
- **Encriptación** de datos en tránsito y en reposo.
- **Auditoría** de las operaciones realizadas sobre la base de datos.
- **Réplica sets** con conmutación automática ante fallos (*failover*) y herramientas de copia de seguridad y restauración.

> **Para saber más.** No te preocupes si algunos de estos términos (sharding, réplica sets, encriptación, roles de usuario) todavía no te dicen gran cosa: son conceptos que se tratan con detalle más adelante, varios de ellos ya en la Unidad de Trabajo 2 (MongoDB Avanzado y Atlas). Aquí solo necesitas una visión de conjunto de todo lo que MongoDB ofrece "de serie".

2.4 Casos de uso de MongoDB

Ideal para…	No recomendado para…
`Aplicaciones web y móviles`	Sistemas transaccionales muy complejos
`Sistemas de contenido y catálogos`	Aplicaciones que requieren joins complejos
`Internet de las cosas (IoT)`	Casos que exigen consistencia fuerte e inmediata
`Plataformas de e-commerce`	Sistemas legacy fuertemente dependientes de SQL
`Sistemas de recomendación`	
`Aplicaciones en tiempo real`	

Como ya se ha comentado, **MongoDB es una excelente opción para muchos casos de uso modernos, pero no es una solución universal**: la elección de la base de datos más adecuada depende siempre de los requisitos concretos del proyecto. Un sistema bancario con operaciones contables estrictamente consistentes seguirá siendo, muy probablemente, un mejor candidato para una base de datos relacional; en cambio, el catálogo de productos de una tienda online —el ejemplo que usaremos durante toda esta unidad— encaja de forma natural con el modelo documental.

2.5 Ecosistema MongoDB

MongoDB no es solo el motor de base de datos: a su alrededor existe todo un ecosistema de herramientas oficiales que iremos conociendo a lo largo del curso.

Herramienta	Qué es
`MongoDB Atlas`	Servicio de base de datos en la nube (DBaaS) con gestión automática de infraestructura. Lo estudiaremos en profundidad en la UT02.
`MongoDB Compass`	Interfaz gráfica oficial para explorar datos, analizar consultas visualmente y depurarlas.
`mongosh`	Shell interactivo de línea de comandos para ejecutar comandos y scripts contra una base de datos MongoDB.
`MongoDB Charts`	Herramienta de visualización de datos y creación de dashboards integrada con Atlas.

Herramienta	Qué es
`MongoDB Realm`	Plataforma para desarrollo móvil con sincronización automática y backend como servicio.
`Drivers oficiales`	Bibliotecas cliente para múltiples lenguajes: Node.js, Python, Java, C#, PHP, Ruby, etc.

En esta unidad trabajaremos principalmente con **mongosh** (para escribir y ejecutar comandos directamente) y, de forma puntual, con **MongoDB Compass** como apoyo visual. En la Unidad de Trabajo 2 pasaremos a trabajar con **MongoDB Atlas**, el servicio en la nube, y con **MongoDB Charts** para la analítica de datos.

Capítulo 3 · Instalación y herramientas de trabajo

El software que necesitas instalar en tu equipo para seguir todas las prácticas del curso, y tus primeros pasos con él.

3.1 Qué vamos a instalar

Para trabajar con MongoDB en local necesitamos, como mínimo, dos piezas de software:

- **MongoDB Community Server**: el propio motor de base de datos, el proceso (`mongod`) que almacena y sirve los datos.
- **mongosh (MongoDB Shell)**: el cliente de línea de comandos que usaremos para conectarnos al servidor y escribir nuestros comandos.
- **MongoDB Compass**: la interfaz gráfica oficial, que permite explorar bases de datos, colecciones y documentos con el ratón, y construir consultas de forma asistida.

> **Para saber más.** Desde la versión 6.0, MongoDB Compass y mongosh se distribuyen de forma independiente del servidor (`mongod`), y es habitual que el propio instalador de MongoDB Community te ofrezca instalarlos a la vez. Comprueba siempre la página oficial de descargas (**www.mongodb.com/try/download**) para obtener la versión más reciente y adecuada a tu sistema operativo.

3.2 Instalación de MongoDB Community Server

El proceso exacto varía según el sistema operativo, pero en todos los casos sigue el mismo esquema general:

- Descargar el instalador correspondiente a tu sistema operativo (Windows, macOS o Linux) desde la web oficial de MongoDB.
- Ejecutar el instalador y, cuando se ofrezca la opción, instalar MongoDB como un **servicio del sistema** (así se iniciará automáticamente con el equipo, sin que tengas que arrancarlo a mano cada vez).
- Marcar la casilla para instalar también **MongoDB Compass** si el instalador la ofrece.
- Comprobar que el servicio está activo antes de intentar conectarte con el shell.

En Windows y macOS, el instalador gráfico se encarga de prácticamente todo el proceso. En distribuciones Linux basadas en Debian/Ubuntu, la instalación suele hacerse añadiendo el repositorio oficial de MongoDB y utilizando el gestor de paquetes del sistema, por ejemplo:

```
// Ejemplo orientativo en Ubuntu/Debian (consulta siempre la guía oficial
// para la versión y el nombre de paquete exactos de tu distribución)
sudo apt-get update
sudo apt-get install -y mongodb-org
sudo systemctl start mongod
sudo systemctl enable mongod
```

3.3 mongosh: el shell interactivo

Una vez instalado y en marcha el servidor, abrimos una terminal y ejecutamos:

```
mongosh
```

Si la conexión se realiza correctamente, veremos un *prompt* similar a este, indicando que estamos conectados a la base de datos `test` (la base de datos por defecto):

```
test>
```

Desde aquí podemos ejecutar comandos de administración y de consulta directamente. Algunos de los primeros comandos útiles para orientarnos son:

Comando	Qué hace
`show dbs`	Lista todas las bases de datos existentes en el servidor
`use nombre_bd`	Cambia (o crea, si no existe) la base de datos activa
`db`	Muestra el nombre de la base de datos activa
`show collections`	Lista las colecciones de la base de datos activa
`db.help()`	Muestra la ayuda de comandos disponibles sobre `db`
`exit`	Cierra la sesión del shell

Idea clave. mongosh es, en realidad, un entorno de **JavaScript**: además de los comandos propios de MongoDB puedes usar variables, bucles, condicionales o funciones de JavaScript estándar. Esto resulta muy útil más adelante para generar datos de prueba o automatizar tareas repetitivas.

3.4 MongoDB Compass: interfaz gráfica

Compass se conecta al mismo servidor que mongosh, pero ofrece una experiencia visual: un panel lateral con las bases de datos y colecciones, un visor de documentos en forma de árbol o de JSON, un constructor de consultas con autocompletado, y herramientas de análisis de rendimiento (entre ellas, una versión visual de `explain()`, que veremos en el Capítulo 7).

Durante el curso trabajaremos principalmente desde **mongosh**, porque escribir los comandos a mano es la mejor forma de interiorizar la sintaxis del lenguaje de consultas de MongoDB (MQL). Sin embargo, es muy recomendable tener Compass abierto en paralelo para **visualizar el resultado** de lo que vamos haciendo, especialmente cuando se trabaja con documentos anidados o colecciones grandes.

3.5 Primeros pasos: conectar y crear una base de datos

MongoDB tiene una particularidad importante frente a SQL: **no es necesario "crear" explícitamente una base de datos ni una colección antes de usarla**. Ambas se crean de forma automática la primera vez que se les inserta un documento. Aun así, el comando `use` es el que nos sitúa en el contexto de trabajo:

```
// Nos situamos en (o creamos) la base de datos tienda_online
use tienda_online

// Mientras no insertemos ningún documento, la base de datos
// todavía NO aparecerá en el listado de show dbs
show dbs

// Al insertar el primer documento, MongoDB crea automáticamente
// tanto la base de datos como la colección si no existían
db.productos.insertOne({ nombre: "Producto de prueba" })

// Ahora sí aparecerá en el listado
show dbs
```

Para saber más. Este comportamiento de "creación perezosa" (solo se crea lo que realmente se usa) es habitual en bases de datos NoSQL y sorprende a quien viene de un entorno SQL, donde siempre hay que ejecutar primero un `CREATE DATABASE` y un `CREATE TABLE` explícitos. En el Capítulo 5 retomaremos esta idea con más detalle.

Capítulo 4 · Tipos de datos en MongoDB (BSON)

MongoDB no almacena JSON "a secas": lo convierte a un formato binario propio, BSON, con más tipos de datos y más eficiente de leer y escribir.

4.1 JSON vs BSON

Cuando escribimos un documento en mongosh, lo escribimos con una sintaxis muy parecida a JSON. Sin embargo, MongoDB no almacena ese JSON como texto: lo convierte internamente a **BSON (Binary JSON)**, una representación binaria que ofrece dos ventajas importantes frente al JSON de texto plano:

- **Más eficiente**: es más rápido de recorrer y de analizar para el motor de base de datos que un texto JSON, y permite saltar directamente a un campo conociendo su posición, sin tener que "leer" todo el documento carácter a carácter.
- **Más tipos de datos**: JSON solo distingue cadenas, números, booleanos, `null`, objetos y arrays. BSON añade tipos adicionales que JSON no tiene, como fechas reales, identificadores únicos (`ObjectId`), datos binarios o números decimales de alta precisión.

Esto significa que, aunque tú escribas y leas tus documentos con una sintaxis parecida a JSON, MongoDB es capaz de distinguir, por ejemplo, entre el número entero `25` y el número `25.0`, o de almacenar una fecha real en lugar de una simple cadena de texto con apariencia de fecha.

4.2 Tipos de datos principales

Tipo BSON	Ejemplo	Equivalente aproximado en SQL
`String`	"Smartphone Pro"	VARCHAR / TEXT
`Number (Int32 / Int64)`	25	INT / BIGINT
`Double`	699.99	FLOAT / DOUBLE
`Decimal128`	NumberDecimal("699.99")	DECIMAL (alta precisión)
`Boolean`	true / false	BOOLEAN
`Date`	ISODate("2026-10-07")	DATE / DATETIME
`Null`	null	NULL
`Array`	["a", "b", "c"]	Sin equivalente directo (requeriría tabla aparte)
`Object / Embedded Document`	{ calle: "...", ciudad: "..." }	Sin equivalente directo (requeriría tabla aparte)
`ObjectId`	ObjectId("6ab6d168...")	Clave primaria autogenerada
`Binary data`	BinData(...)	BLOB

Los dos últimos tipos de la tabla —**Array** y **Object**— son los que marcan de verdad la diferencia con el modelo relacional: permiten representar listas y estructuras anidadas **dentro de un mismo documento**, sin necesidad de tablas relacionadas. Volveremos sobre esta idea en el Capítulo 6, al

trabajar con operadores sobre arrays, y la ampliaremos en la Unidad de Trabajo 2 al hablar de modelado de datos.

4.3 El campo _id y el tipo ObjectId

Todo documento de MongoDB tiene un campo `_id` que actúa como su **clave primaria**: identifica al documento de forma única dentro de la colección y es, además, el primer campo del documento y el único sobre el que existe siempre un índice (lo veremos en el Capítulo 7).

Si no especificamos un `_id` al insertar un documento, MongoDB genera automáticamente uno de tipo `ObjectId`: un identificador de 12 bytes construido a partir de la marca de tiempo de creación, un identificador de máquina/proceso y un contador, lo que garantiza (en la práctica) que nunca se repita.

```
// Si no indicamos _id, MongoDB lo genera automáticamente
db.productos.insertOne({ nombre: "Monitor 24" })
// -> { acknowledged: true, insertedId: ObjectId('6ab6d168bd20a8df07621005') }

// También podemos especificar manualmente un _id propio
db.productos.insertOne({
  _id: "PROD-001",
  nombre: "Monitor 24",
  categoria: "electronica"
})
```

Atención. Un `_id` asignado manualmente debe ser único dentro de su colección: si intentas insertar dos documentos con el mismo `_id`, MongoDB rechazará el segundo con un error de clave duplicada. En la mayoría de los casos (y en todos los ejemplos de este manual, salvo que se indique lo contrario) dejaremos que MongoDB genere el `ObjectId` automáticamente.

4.4 Fechas, decimales y otros tipos especiales

Fechas (Date)

A diferencia de guardar una fecha como una simple cadena de texto (`"2026-10-07"`), el tipo `Date` de MongoDB almacena un instante real (fecha y hora, en UTC internamente), lo que permite hacer comparaciones y cálculos correctos sobre fechas:

```
db.productos.updateOne(
  { nombre: "Laptop Elite" },
  { $set: { fecha_actualizacion: new Date() } }
)
```

Números decimales de alta precisión (Decimal128)

El tipo `Double` estándar de BSON (igual que el `float`/`double` de la mayoría de lenguajes de programación) puede introducir pequeños errores de redondeo en operaciones con decimales, algo habitual y esperable en informática (por ejemplo, al calcular un precio medio puede aparecer un resultado como `27.490000000000002` en lugar de `27.49`, que es simplemente cómo se representan internamente los números en coma flotante). Para casos donde la precisión decimal exacta es crítica —como los importes monetarios en un sistema financiero real— MongoDB ofrece el tipo `Decimal128`.

> **Para saber más.** Para el trabajo habitual de esta unidad usaremos números normales (`Double`) para precios y cantidades: es más que suficiente para nuestros catálogos de ejemplo, y es importante que te familiarices con este pequeño "ruido" de redondeo, porque lo verás aparecer de forma natural en los resultados de algunas agregaciones (framework que estudiaremos en la Unidad de Trabajo 2).

Arrays y documentos embebidos

Ya los hemos visto en el ejemplo del Capítulo 2: un array permite guardar una lista de valores (`caracteristicas: ["6.7 OLED", "128GB", "5G"]`) y un documento embebido permite anidar un objeto completo dentro de otro (`direccion: { calle: ..., ciudad: ... }`). Ambos tipos son fundamentales en MongoDB y los usaremos constantemente a partir del Capítulo 6.

Capítulo 5 · Bases de datos y colecciones

Cómo se crean, organizan y administran las bases de datos y colecciones con las que trabajaremos durante todo el curso.

5.1 Creación "perezosa" de bases de datos

Como se ha adelantado en el Capítulo 3, MongoDB no exige crear explícitamente una base de datos antes de usarla. El comando `use` selecciona una base de datos como "activa" para los siguientes comandos, y la crea de forma transparente la primera vez que se le añade algún dato:

```
use tienda_online
// A partir de aquí, todos los comandos db.coleccion.xxx()
// actuarán sobre la base de datos tienda_online
```

Esta será la base de datos de ejemplo que usaremos en el resto del manual: **tienda_online**, con una colección principal llamada **productos**, igual que en las prácticas que hemos ido haciendo en clase.

5.2 Creación de colecciones

Igual que ocurre con las bases de datos, una colección se crea automáticamente al insertar en ella el primer documento (**creación implícita**). Sin embargo, en ocasiones interesa crear la colección de forma **explícita**, sobre todo cuando queremos aplicarle alguna opción especial desde el principio, como un límite de tamaño o un validador de esquema.

```
// Creación implícita: basta con insertar
db.productos.insertOne({ nombre: "Producto de prueba" })

// Creación explícita, sin insertar nada todavía
db.createCollection("pedidos")

// Creación explícita con opciones (ejemplo: colección "capped",
// de tamaño fijo, útil para registros tipo log)
db.createCollection("logs", { capped: true, size: 100000 })
```

> **Para saber más.** La validación de esquemas con `$jsonSchema` —que permite exigir, por ejemplo, que todo documento de una colección tenga obligatoriamente un campo `precio` de tipo numérico— se trata en profundidad en la Unidad de Trabajo 2, ya que requiere usar `db.createCollection()` con opciones avanzadas o `collMod` sobre una colección existente.

5.3 Operaciones de administración sobre colecciones

Comando	Qué hace
`show collections`	Lista las colecciones de la base de datos activa
`db.coleccion.stats()`	Muestra estadísticas de la colección (tamaño, nº de documentos, índices…)
`db.coleccion.renameCollection("nuevo_nombre")`	Renombra la colección
`db.coleccion.drop()`	Elimina la colección por completo, con todos sus documentos e índices
`db.dropDatabase()`	Elimina la base de datos activa por completo

> **Atención.** `drop()` y `dropDatabase()` son operaciones **irreversibles**. Antes de ejecutarlas en una colección o base de datos real conviene asegurarse de estar conectado a la base de datos correcta (`db` para comprobarlo) y, si los datos son importantes, haber hecho antes una copia de seguridad (lo veremos en el Capítulo 9).

5.4 Namespaces: cómo se identifica una colección

Internamente, MongoDB identifica cada colección mediante su **namespace**: la combinación `nombre_de_la_base_de_datos.nombre_de_la_coleccion`. Por ejemplo, la colección de productos de nuestra tienda se identifica como `tienda_online.productos`. Este namespace debe ser único en todo el servidor, y es la razón por la que dos colecciones con el mismo nombre pueden coexistir sin problema si pertenecen a bases de datos distintas.

```
// Ver el namespace completo de la colección activa
db.productos.getFullName()
// -> tienda_online.productos
```

5.5 El esquema flexible en la práctica

Una de las diferencias más visibles frente a SQL es que, dentro de una misma colección, **no todos los documentos tienen por qué compartir exactamente los mismos campos**. MongoDB no lo impide por defecto (aunque, como hemos visto, sí se puede forzar un esquema con validadores si se desea).

```
db.productos.insertMany([
  { nombre: "Camiseta Algodón", categoria: "ropa", precio: 19.99 },
  { nombre: "Zapatillas Running", categoria: "deportes", precio: 89.99, marca:
"SportMax" }
])
// El primer documento no tiene campo "marca" y es perfectamente válido
```

Idea clave. Esta flexibilidad es muy potente, pero también exige disciplina por nuestra parte: si distintos documentos de la misma colección usan nombres de campo ligeramente distintos para lo mismo (por ejemplo, `categoria` en unos y `categoría` —con tilde— en otros), nuestras consultas pueden devolver resultados incompletos sin que MongoDB avise de ningún error, porque desde su punto de vista son simplemente dos campos distintos. Mantener nombres de campo consistentes dentro de una misma colección es una buena práctica fundamental, y es precisamente uno de los errores más habituales que hemos visto al corregir vuestra Actividad 2 de CRUD.

Capítulo 6 · Operaciones CRUD

Las cuatro operaciones básicas de persistencia de datos: crear, leer, actualizar y eliminar documentos. Es, con diferencia, el capítulo más importante del manual: todo lo que hagamos el resto del curso se apoya en estas operaciones.

6.1 ¿Qué es CRUD?

CRUD es el acrónimo de las cuatro operaciones básicas de persistencia de datos que cualquier sistema de gestión de bases de datos debe ofrecer:

Operación	Significado	Equivalente en SQL
`Create`	Crear nuevos documentos	INSERT
`Read`	Leer y consultar documentos	SELECT
`Update`	Actualizar documentos existentes	UPDATE
`Delete`	Eliminar documentos	DELETE

A lo largo de este capítulo trabajaremos siempre sobre la misma base de datos de ejemplo, **tienda_online**, y sobre su colección principal, **productos**. Un documento típico de esta colección tiene la siguiente forma:

```
{
  "nombre": "Laptop Gaming",
  "categoria": "electronica",
  "precio": 999.99,
  "stock": 15,
  "marca": "TechBrand",
  "caracteristicas": ["16GB RAM", "512GB SSD", "RTX 3060"]
}
```

6.2 CREATE: crear documentos

MongoDB ofrece dos métodos principales para insertar documentos en una colección: `insertOne()` para insertar un único documento, e `insertMany()` para insertar varios de una sola vez.

insertOne()

```
db.productos.insertOne({
  nombre: "Smartphone Pro",
  categoria: "electronica",
  precio: 699.99,
  stock: 25,
  marca: "PhoneTech",
  caracteristicas: ["6.7 OLED", "128GB", "5G"]
})
```

Si la inserción tiene éxito, MongoDB devuelve un documento de confirmación con `acknowledged: true` y el `insertedId` generado (o el que hayamos indicado nosotros):

```
{
  acknowledged: true,
  insertedId: ObjectId('6ab6d168bd20a8df07621005')
}
```

insertMany()

Para insertar varios documentos a la vez se le pasa un **array** de documentos:

```
db.productos.insertMany([
  {
    nombre: "Tablet Lite",
    categoria: "electronica",
    precio: 299.99,
    stock: 30,
    marca: "TabTech",
    caracteristicas: ["10.1\"", "64GB", "WiFi"]
  },
  {
    nombre: "Auriculares Inalambricos",
    categoria: "accesorios",
    precio: 89.99,
    stock: 50,
    marca: "SoundMax",
    caracteristicas: ["Noise Cancelling", "20h bateria"]
  }
])
```

El campo _id (recordatorio)

Como vimos en el Capítulo 4, todo documento tiene un `_id` que actúa como identificador único: si no lo especificamos, MongoDB genera automáticamente un `ObjectId`. También podemos especificarlo manualmente, siempre que sea único dentro de la colección:

```
db.productos.insertOne({
  _id: "PROD-001",
  nombre: "Monitor 24",
  categoria: "electronica",
  precio: 199.99,
  stock: 10
})
```

6.3 READ: consultar documentos

find() y findOne()

El método `find()` es, sin duda, el que más usaremos durante todo el curso. Devuelve **todos** los documentos que cumplen un criterio (o todos, si no se indica ninguno); `findOne()` devuelve únicamente el primero que encuentra.

```
// Todos los productos
db.productos.find()

// Un producto cualquiera
db.productos.findOne()

// Productos de electronica
db.productos.find({ categoria: "electronica" })
```

Operadores de comparación

Para construir condiciones más allá de la simple igualdad, MongoDB ofrece un conjunto de **operadores de comparación**, que siempre se escriben como una clave que empieza por `$` dentro del valor del campo:

Operador	Descripción	Ejemplo
`$eq`	Igual a	{ precio: { $eq: 100 } }
`$ne`	No igual	{ categoria: { $ne: "electronica" } }
`$gt`	Mayor que	{ precio: { $gt: 500 } }
`$gte`	Mayor o igual	{ stock: { $gte: 10 } }
`$lt`	Menor que	{ precio: { $lt: 100 } }
`$lte`	Menor o igual	{ stock: { $lte: 5 } }
`$in`	En array	{ categoria: { $in: ["electronica"] } }
`$nin`	No en array	{ marca: { $nin: ["MarcaA"] } }

Estos operadores se pueden combinar dentro de un mismo campo (ambas condiciones deben cumplirse, ya que están dentro del mismo objeto) para expresar rangos:

```
// Productos entre 100 y 500 euros
db.productos.find({
  precio: { $gte: 100, $lte: 500 }
})

// Productos de electronica o accesorios
db.productos.find({
```

```
  categoria: { $in: ["electronica", "accesorios"] }
})
```

Operadores lógicos

Operador	Descripción	Ejemplo
$and	Y lógico (todas las condiciones)	{ $and: [{cond1}, {cond2}] }
$or	O lógico (al menos una condición)	{ $or: [{cond1}, {cond2}] }
$not	Negación de una condición	{ precio: { $not: { $gt: 100 } } }
$nor	Ni una condición ni la otra	{ $nor: [{cond1}, {cond2}] }

```
// Productos de electronica con precio > 500
db.productos.find({
  $and: [
    { categoria: "electronica" },
    { precio: { $gt: 500 } }
  ]
})

// Productos de electronica O precio < 100
db.productos.find({
  $or: [
    { categoria: "electronica" },
    { precio: { $lt: 100 } }
  ]
})
```

Idea clave. Cuando varias condiciones van simplemente **separadas por comas dentro del mismo objeto** `{ }`, MongoDB las combina de forma implícita con un "Y" lógico, igual que un `$and`. Por eso en la mayoría de consultas sencillas no hace falta escribir `$and` de forma explícita: solo es necesario cuando queremos combinar varias condiciones **sobre el mismo campo** (algo que, de lo contrario, se sobrescribiría) o cuando el criterio exige explícitamente mezclar `$and` con `$or`.

Proyección: seleccionar campos

La **proyección** es el segundo argumento (opcional) de `find()`: permite indicar qué campos del documento queremos incluir o excluir del resultado, lo cual es muy recomendable por rendimiento cuando no necesitamos el documento completo.

```
// Incluir solo nombre y precio (con 1 indicamos "incluir")
db.productos.find(
  { categoria: "electronica" },
  { nombre: 1, precio: 1, _id: 0 }
)

// Excluir un campo concreto (con 0 indicamos "excluir")
```

```
db.productos.find(
  { activo: true },
  { caracteristicas: 0, fecha_creacion: 0 }
)
```

Atención. No se pueden mezclar inclusiones y exclusiones en la misma proyección (salvo el caso especial de `_id`, que siempre se puede excluir con `_id: 0` aunque el resto de campos estén en modo "inclusión"). O decides qué campos **quieres ver** (todo 1), o decides qué campos **quieres ocultar** (todo 0), pero no ambas cosas a la vez.

Métodos útiles para consultas: sort(), limit() y count()

Método	Qué hace
`sort({ campo: 1 })`	Ordena de forma ascendente por el campo indicado
`sort({ campo: -1 })`	Ordena de forma descendente por el campo indicado
`limit(n)`	Limita el número de resultados devueltos a n
`skip(n)`	Salta los primeros n resultados (útil para paginación)
`count() / countDocuments()`	Cuenta el número de documentos que cumplen el criterio

Estos métodos se **encadenan** después de `find()`, y se pueden combinar entre sí:

```
// Los 5 productos mas caros
db.productos.find()
  .sort({ precio: -1 })
  .limit(5)
```

Idea clave. El orden en que se escriben `sort()` y `limit()` no cambia el resultado (MongoDB siempre ordena antes de limitar), pero sí es importante no olvidar nunca el `sort()` cuando se usa `limit()`: sin un criterio de ordenación explícito, MongoDB no garantiza en qué orden se devuelven los documentos, por lo que "los 5 primeros" podría no significar lo que esperamos.

6.4 UPDATE: actualizar documentos

MongoDB ofrece tres métodos principales para modificar documentos existentes:

Método	Qué hace
`updateOne(filtro, cambios)`	Actualiza el primer documento que cumple el filtro
`updateMany(filtro, cambios)`	Actualiza todos los documentos que cumplen el filtro
`replaceOne(filtro, documento)`	Sustituye el documento completo por uno nuevo

Atención. Es muy importante no confundir el **filtro** (qué documentos seleccionar, igual que en `find()`) con el objeto de **cambios** (qué modificar en ellos). Son siempre dos argumentos distintos: `updateOne(filtro, cambios)`. Si por error se combinan ambos en un único objeto, o se pasa el

segundo filtro como si fuera un argumento adicional en vez de parte del mismo objeto de filtro, MongoDB no lanzará necesariamente un error, pero el resultado no será el esperado: una condición puede quedar "silenciosamente" sin aplicarse.

updateOne()

```
db.productos.updateOne(
  { nombre: "Laptop Gaming" },
  { $set: { precio: 1099.99 } }
)
```

La respuesta de una operación de actualización indica cuántos documentos coincidieron con el filtro (`matchedCount`) y cuántos se modificaron realmente (`modifiedCount`; puede ser menor que `matchedCount` si el valor ya era el mismo):

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Operadores de actualización

Operador	Descripción	Ejemplo
`$set`	Establece el valor de un campo (lo crea si no existe)	{ $set: { precio: 100 } }
`$unset`	Elimina un campo del documento	{ $unset: { caracteristicas: "" } }
`$inc`	Incrementa (o decrementa) un valor numérico	{ $inc: { stock: 5 } }
`$mul`	Multiplica un valor numérico	{ $mul: { precio: 1.1 } }
`$push`	Añade un elemento a un array (permite duplicados)	{ $push: { tags: "nuevo" } }
`$addToSet`	Añade un elemento a un array solo si no existe ya	{ $addToSet: { tags: "popular" } }
`$pull`	Elimina de un array todos los elementos que cumplan una condición	{ $pull: { tags: "antiguo" } }

```
// Aplicar un descuento del 10% a toda la electronica
db.productos.updateMany(
  { categoria: "electronica" },
  { $mul: { precio: 0.9 } }
)
```

```
// Marcar como inactivos los productos sin stock
db.productos.updateMany(
  { stock: 0 },
  { $set: { activo: false } }
)
```

Modificar arrays: $push y $pull

```
// Agregar una característica nueva
db.productos.updateOne(
  { nombre: "Tablet Vision" },
  { $push: { caracteristicas: "GPS" } }
)

// Quitar una característica concreta
db.productos.updateOne(
  { nombre: "Tablet Vision" },
  { $pull: { caracteristicas: "4G" } }
)
```

Idea clave. `$push` admite duplicados (si el valor ya estaba en el array, se añadirá una segunda vez); `$addToSet` lo evita, comprobando primero si el valor ya existe en el array. Elige uno u otro según si tiene sentido que un mismo valor pueda repetirse en tu caso concreto.

6.5 DELETE: eliminar documentos

Método	Qué hace
`deleteOne(filtro)`	Elimina el primer documento que cumple el filtro
`deleteMany(filtro)`	Elimina todos los documentos que cumplen el filtro

Atención. Las operaciones de eliminación son **destructivas e irreversibles**: una vez ejecutado un `deleteMany()`, los documentos eliminados no se pueden recuperar (salvo que dispongamos de una copia de seguridad previa, ver Capítulo 9). Por eso la primera buena práctica de este apartado es: **comprueba siempre tu filtro con un `find()` antes de convertirlo en un `delete`**.

```
// Eliminar un producto específico
db.productos.deleteOne({ nombre: "Producto Viejo" })

// Eliminar productos sin precio
db.productos.deleteMany({ precio: 0 })

// Verificar ANTES de eliminar: primero comprobamos qué se borraría...
db.productos.find({ activo: false })
// ...y solo entonces ejecutamos el borrado
```

```
db.productos.deleteMany({ activo: false })
```

6.6 Buenas prácticas y resumen de comandos CRUD

- **Usa siempre `_id`** para las operaciones sobre un documento muy concreto: es el campo más rápido de buscar porque siempre tiene un índice.
- **Prueba tus consultas con `find()` antes de convertirlas en `update` o `delete`**: comprobar primero qué documentos selecciona un filtro evita modificar o borrar más (o menos) de lo esperado.
- **Usa proyección** en tus consultas de lectura para pedir solo los campos que realmente necesitas: reduce el tráfico de red y mejora el rendimiento, especialmente en documentos grandes.
- **Mantén nombres de campo consistentes** dentro de una misma colección (sin mezclar, por ejemplo, `categoria` y `categoría`): es el error más habitual y más difícil de detectar, porque no produce ningún mensaje de error.
- **Recuerda que el filtro y los cambios son siempre dos argumentos distintos** en `updateOne`/`updateMany`/`deleteOne`/`deleteMany`: combinar dos condiciones en dos objetos separados donde se esperaba uno solo no da error, pero hace que la segunda condición se ignore silenciosamente.
- **Considera crear índices** sobre los campos que consultes con más frecuencia: lo veremos en detalle en el Capítulo 7.

> **Para saber más.** **Recordatorio importante sobre atomicidad**: en MongoDB, las operaciones sobre un único documento son siempre **atómicas** (se aplican por completo o no se aplican en absoluto, sin estados intermedios). Sin embargo, una operación como `updateMany()`, que afecta a varios documentos, **no es atómica como conjunto**: si el proceso se interrumpe a mitad de camino, es posible que algunos documentos ya se hayan actualizado y otros no. Para operaciones que deben tratarse como "todo o nada" sobre varios documentos a la vez existen las **transacciones multi-documento**, que se estudian en la Unidad de Trabajo 2.

Operación	Métodos	Operadores / claves más usados
`Create`	insertOne(), insertMany()	_id
`Read`	find(), findOne()	$eq, $gt, $gte, $lt, $lte, $in, $nin, $and, $or
`Update`	updateOne(), updateMany(), replaceOne()	$set, $unset, $inc, $mul, $push, $addToSet, $pull
`Delete`	deleteOne(), deleteMany()	Filtros de consulta (igual que en find)

Capítulo 7 · Índices y rendimiento de consultas

Cómo acelerar nuestras consultas indicando a MongoDB dónde buscar, en lugar de obligarlo a recorrer la colección entera cada vez. Es la herramienta que marca la diferencia entre una aplicación que "funciona" y una que funciona bien con datos reales.

7.1 ¿Qué es un índice en MongoDB?

Un **índice** es una estructura de datos adicional que mejora la velocidad de las consultas sobre un campo determinado, a costa de espacio en disco extra y de cierto mantenimiento adicional en cada escritura. Funciona exactamente igual que el índice alfabético de un libro: permite encontrar rápidamente los documentos que contienen un valor concreto, sin tener que revisar cada documento de la colección uno por uno.

Sin un índice sobre un campo, MongoDB debe hacer un **escaneo completo de la colección** (*collection scan* o `COLLSCAN`) para resolver una consulta sobre ese campo: examina todos y cada uno de los documentos para comprobar si cumplen el criterio.

7.2 Índices como árboles B (B-Tree)

MongoDB implementa sus índices utilizando **árboles B (B-Tree)**, una estructura de árbol balanceado ampliamente usada en bases de datos de todo tipo. En un B-Tree, las claves (los valores del campo indexado) se mantienen siempre ordenadas, lo que permite búsquedas binarias eficientes que reducen drásticamente el número de comparaciones necesarias.

- La búsqueda en un índice B-Tree tiene una complejidad aproximada de **O(log n)**, frente a la **O(n)** de una búsqueda sin índice (es decir, un escaneo completo).
- Los nodos del árbol apuntan a hojas que referencian a los documentos reales de la colección: una vez localizada la clave en el índice, se accede directamente al documento correspondiente.

Idea clave. No necesitas saber implementar un árbol B para usar MongoDB con soltura, pero sí conviene entender **por qué** un índice acelera tanto las búsquedas: O(log n) crece muchísimo más despacio que O(n) a medida que aumenta el número de documentos. En una colección con un millón de documentos, una búsqueda sin índice puede necesitar examinar hasta un millón de documentos; con un buen índice, puede resolverse en apenas 20 comparaciones.

7.3 El índice por defecto y los índices únicos

Cada colección en MongoDB tiene automáticamente un índice B-Tree en el campo `_id`, lo que garantiza búsqueda rápida por `_id` desde el primer momento. Este índice, además, es siempre **único**: MongoDB impone que no pueda haber dos documentos con el mismo `_id` en la misma colección.

Más allá del `_id`, podemos crear índices adicionales sobre cualquier otro campo, y configurarlos también como únicos si queremos garantizar, por ejemplo, que no existan dos usuarios con el mismo correo electrónico:

```
db.usuarios.createIndex(
  { email: 1 },
  { unique: true }
)
// A partir de ahora, insertar un email ya existente lanzará un error
```

Atención. Un índice único **falla** al crearse si la colección ya contiene documentos duplicados en ese campo: hay que limpiar los datos existentes antes de poder crear el índice.

7.4 Costo de los índices

- Los índices aceleran las **lecturas**, pero añaden sobrecarga a las **escrituras** (insertar, actualizar o eliminar un documento obliga a actualizar también todos sus índices).
- Cada índice ocupa espacio adicional en **disco y en memoria**; mantener los índices más usados en RAM mejora mucho el rendimiento.
- Tener **demasiados índices** en una colección puede degradar el rendimiento de escritura: lo recomendable es indexar solo lo necesario (ver la regla ESR, más adelante).
- El uso real de un índice se puede comprobar con `explain()` (apartado 7.7) o con estadísticas de uso de índices.

7.5 Limitaciones de los índices

Límite	Valor
Número máximo de índices por colección	64
Campos máximos en un índice compuesto	32
Longitud máxima de una clave indexada	1024 bytes
Campos de tipo array por índice compuesto	1 (no se permite más de un array indexado a la vez)

Además de los índices "normales", existen tipos especiales: los índices **geoespaciales** (`2d`, `2dsphere`) para datos de coordenadas, y los índices **hashed**, usados para distribuir datos de forma uniforme en un clúster con sharding (lo veremos en la Unidad de Trabajo 2).

7.6 Tipos de índices y su creación

El comando general para crear un índice es:

```
db.<coleccion>.createIndex({ campo: orden }, { opciones })
// orden: 1 para ascendente, -1 para descendente
```

Índices de campo único

El tipo más básico: un índice sobre un solo campo, que acelera las búsquedas que filtran exactamente por ese campo (se puede indexar también un campo anidado usando notación de punto):

```
db.productos.createIndex({ categoria: 1 })
db.usuarios.createIndex({ 'direccion.ciudad': 1 })
```

Para saber más. Un índice de campo único solo optimiza consultas por ese campo exacto, no combinaciones con otros campos (para eso hace falta un índice compuesto).

Índices compuestos

Un índice compuesto incluye más de un campo, y es muy útil para consultas que combinan varios criterios de filtro y/o de ordenación:

```
db.productos.createIndex({ categoria: 1, precio: -1 })
```

El **orden de los campos en el índice es crítico**: el índice anterior puede usarse para consultas por `categoria` sola, o por `categoria` + `precio`, pero no de forma eficiente para consultas solo por `precio`. Un buen diseño de índice compuesto sigue la **regla ESR** (apartado 7.8).

Índices multiclave (arrays)

Si el campo indexado es un **array**, MongoDB indexa automáticamente cada elemento del array por separado (cada valor actúa como una entrada independiente en el índice). A estos índices se les llama **multiclave**:

```
db.productos.createIndex({ caracteristicas: 1 })
// find({ caracteristicas: 'GPS' }) usará este índice
// para encontrar cualquier documento cuyo array contenga 'GPS'
```

Atención. Limitación importante: no se permite más de un campo de tipo array en un mismo índice compuesto (MongoDB no puede indexar el producto cartesiano de dos arrays a la vez). Además, los índices multiclave pueden aumentar bastante el tamaño del índice, ya que un solo documento genera una entrada por cada elemento de su array.

Índices de texto

Pensados específicamente para búsqueda de texto por palabras clave (búsqueda "textual", no exacta):

```
db.articulos.createIndex({ descripcion: 'text' })

db.articulos.find({
  $text: { $search: 'palabra clave' }
})
```

Una colección puede tener como máximo **un único índice de texto**, aunque ese índice puede combinar varios campos a la vez. También se puede especificar el idioma (por ejemplo, `{ default_language: 'spanish' }`) para que el motor de texto aplique correctamente las reglas de *stemming* y las palabras vacías de ese idioma.

Índices TTL (Time To Live)

Un índice especial sobre un campo de fecha que elimina automáticamente los documentos transcurrido cierto tiempo. Muy útil para datos temporales como registros de log o sesiones que deben expirar:

```
db.logs.createIndex(
  { fecha: 1 },
  { expireAfterSeconds: 3600 }
)
// Los documentos se eliminarán una hora después de su campo "fecha"
```

Índices parciales

Indexan únicamente los documentos que cumplen un filtro dado, lo que reduce el tamaño del índice y mejora el rendimiento cuando solo un subconjunto de documentos es realmente relevante para las consultas que lo usarán:

```
db.personas.createIndex(
  { edad: 1 },
  { partialFilterExpression: { edad: { $gte: 18 } } }
)
```

Tipo de índice	Uso principal
`Campo único`	Indexar un campo individual para búsquedas directas
`Compuesto`	Indexar combinación de campos (consultas por varios campos)
`Multiclave`	Indexar campos que contienen arrays (cada elemento indexado)
`Texto`	Búsqueda por palabras clave en campos textuales

Tipo de índice	Uso principal
`TTL`	Eliminación automática de documentos antiguos
`Parcial`	Indexar solo un subconjunto de documentos que cumple una condición
`Único`	Garantizar unicidad de valores en el campo indexado

7.7 Análisis de consultas con explain()

`explain()` permite analizar cómo el motor de MongoDB ejecuta realmente una consulta u operación (`find`, `aggregate`, `update`…). Es la herramienta esencial para optimizar el rendimiento: nos dice si una consulta usó algún índice o si hizo un escaneo completo, cuántos documentos examinó y cuánto tardó.

```
db.productos.find({
  categoria: "electronica"
}).explain('executionStats')
```

`explain()` admite varios **modos de verbosidad**:

Modo	Descripción
`queryPlanner`	Muestra el plan de ejecución elegido, sin ejecutar la consulta de verdad (es el modo por defecto)
`executionStats`	Ejecuta la consulta y añade estadísticas reales de ejecución (tiempo, documentos examinados…)
`allPlansExecution`	Como executionStats, pero incluyendo detalles de todos los planes alternativos considerados

Interpretando la salida de explain()

- **queryPlanner**: detalles del plan escogido (y de las alternativas, si las hubo).
- **winningPlan**: el plan de ejecución finalmente seleccionado por el optimizador. Dentro de él, el campo `stage` indica la etapa principal: `COLLSCAN` para escaneo completo, o `IXSCAN` si se usó un índice (en cuyo caso aparecerán detalles como `indexName` y `indexBounds`, el rango de búsqueda dentro del índice).
- **executionStats**: estadísticas posteriores a la ejecución real, entre ellas `nReturned` (documentos devueltos), `totalDocsExamined` (total de documentos examinados: debería ser igual a `nReturned` si se usó un índice muy selectivo, y mucho mayor si hubo `COLLSCAN`), y `executionTimeMillis` (tiempo que tardó la operación, en milisegundos).

Ejemplo: índice vs. escaneo completo

Supongamos la consulta `db.productos.find({ precio: 100 })` sobre una colección con **N** documentos en total, de los cuales **M** tienen `precio = 100`:

- **Sin índice** en `precio`: `stage: 'COLLSCAN'`, `totalDocsExamined = N` (todos los documentos de la colección), `nReturned = M`.
- **Con índice** en `precio`: `stage: 'IXSCAN'` usando ese índice, `totalDocsExamined = M` (solo los documentos con `precio = 100`), `nReturned = M`.

La diferencia es clara: el índice evitó examinar los `N - M` documentos irrelevantes, reduciendo drásticamente el trabajo y el tiempo de la consulta. Esta comparación —`totalDocsExamined` frente a `nReturned`— es, en la práctica, la forma más directa de saber si una consulta concreta se beneficiaría de un nuevo índice.

> **Idea clave.** Si ves `COLLSCAN` donde esperabas `IXSCAN`, considera crear un índice para ese filtro. Y si `totalDocsExamined` es mucho mayor que `nReturned` aun usando un índice, puede que la consulta esté leyendo demasiados documentos "de más": a veces hay que reescribir la consulta para que aproveche mejor el índice existente (por ejemplo, respetando el mismo orden de campos que el índice compuesto), o combinar condiciones en una sola consulta en lugar de varias.

7.8 Mejores prácticas: la regla ESR

Un buen diseño de índices es tan importante como el propio modelado de los datos: índices bien diseñados garantizan que las consultas habituales sean eficientes, reduciendo la latencia y la carga de CPU/IO; un mal diseño (índices faltantes, o en el orden incorrecto) puede provocar que incluso bases de datos pequeñas tengan bajo rendimiento.

La **regla ESR (Equality, Sort, Range · Igualdad, Orden, Rango)** es una guía fundamental para construir índices compuestos eficaces: sugiere el orden en que deben colocarse los campos de un índice compuesto.

1. Primero, los campos por los que se consulta con **igualdad** (`==`, un valor exacto).
2. Después, los campos por los que se realiza **ordenamiento** (`sort`) en la consulta.
3. Por último, los campos que se consultan por **rango** (`$gt`, `$lt`, `$in`, etc.).

Siguiendo este orden, el índice podrá soportar eficientemente las tres partes de la consulta: las búsquedas por igualdad aprovechan la parte inicial del índice; con el índice ya ordenado según el siguiente campo, la consulta puede evitar un `sort` adicional en memoria; y, finalmente, los filtros por rango se aplican sobre el subconjunto ya filtrado y ordenado.

Ejemplo de aplicación de ESR

Consulta frecuente: obtener usuarios **activos** (`estado: 'activo'`) de cierta **edad mínima** (`edad > 30`) ordenados por **apellido**.

- Filtro de igualdad: `estado = 'activo'`.
- Ordenamiento: por `apellido` ascendente.
- Filtro de rango: `edad > 30`.

Según ESR, un índice ideal sería:

```
db.usuarios.createIndex(
  { estado: 1, apellido: 1, edad: 1 }
)
```

Es decir: `estado` (igualdad) primero, `apellido` (sort) segundo, `edad` (rango) al final. Este índice permitirá buscar rápidamente usuarios activos, ya ordenados por apellido, y luego filtrar por el rango de edad.

Selectividad y composición de índices

- **Selectividad**: un índice es más útil cuando filtra una fracción pequeña de la colección. Campos con valores muy repetidos (por ejemplo, un campo booleano) no son buenos candidatos, ya que el índice no reduce mucho la búsqueda.
- **Índices compuestos vs. múltiples índices simples**: en lugar de crear muchos índices simples para distintos campos, a menudo conviene un índice compuesto que abarque varios campos usados juntos en consultas. Un índice compuesto puede servir para consultas por el primer campo solo, o por el primero y segundo juntos, etc. En cambio, múltiples índices separados no pueden combinarse fácilmente en un solo plan eficiente (MongoDB suele usar solo un índice por consulta).

Evitar la sobreindexación

- **Índices innecesarios**: tener índices que no se usan en consultas reales solo añade sobrecarga en escrituras y consume memoria. Conviene monitorizar el uso de índices y eliminar los que no sean útiles.
- **Índices redundantes**: evita índices duplicados o que sean prefijo de otros. Por ejemplo, si ya existe el índice `{estado:1, apellido:1}`, un índice adicional `{estado:1}` es redundante, porque las consultas por `estado` ya pueden usar el índice compuesto.
- **Demasiados índices**: cada índice extra ralentiza inserciones y actualizaciones. Prioriza los índices que benefician a las consultas más críticas de tu aplicación, y reevalúa periódicamente la estrategia de índices conforme cambien los patrones de acceso a los datos.

> **Para saber más.** Se puede consultar el uso real de los índices de una colección con `db.nombreColeccion.aggregate([ { $indexStats: {} } ])`, que informa de cuántas veces se ha usado cada índice desde el último reinicio del servidor: una forma objetiva de detectar índices que "sobran".

Capítulo 8 · Exportación e importación de datos

Cómo mover datos dentro y fuera de MongoDB usando archivos JSON y CSV, sin necesidad de escribir comandos de inserción uno a uno.

8.1 ¿Por qué exportar e importar datos?

Hasta ahora hemos insertado documentos escribiendo directamente `insertOne()` o `insertMany()` en mongosh. Esto es perfecto para practicar, pero en un caso real los datos casi siempre llegan (o tienen que salir) en forma de **archivos**: un listado de productos en Excel/CSV que nos pasa el departamento de compras, una copia de los datos para analizarlos en otra herramienta, o un traspaso de información entre un entorno de pruebas y uno de producción.

MongoDB incluye dos utilidades de línea de comandos pensadas exactamente para esto: **mongoimport** y **mongoexport**. A diferencia de mongosh, estas herramientas no se ejecutan dentro del shell, sino directamente desde la terminal del sistema operativo (por eso sus nombres no empiezan por `db.`).

A partir de la versión 4.4 de MongoDB Server estas herramientas ya no vienen incluídas por defecto en el paquete, por lo que hay que descargarlas desde el Centro de Descargas de MongoDB: https://www.mongodb.com/try/download/database-tools

> **Para saber más.** No confundas `mongoimport`/`mongoexport` (pensados para archivos JSON/CSV "legibles", documento a documento) con `mongodump`/`mongorestore` (pensados para copias de seguridad binarias completas de una base de datos). Veremos estos últimos en el Capítulo 9.

8.2 mongoimport: cargar datos desde un archivo

`mongoimport` lee un archivo externo y crea un documento en una colección por cada registro que encuentra. Su sintaxis básica es:

```
mongoimport --db=tienda_online --collection=productos --file=productos_import.json
```

Importa el contenido de productos.json en la colección productos de la base de datos tienda_online.

Si queremos importar los datos contenidos en un array tendremos que agregar el parámetro –jsonArray:

```
mongoimport --db=tienda_online --collection=productos –jsonArray --
file=productos_array_import.json
```

Las opciones más habituales de `mongoimport` son:

Opción	Qué hace
`--db`	Base de datos de destino
`--collection`	Colección de destino
`--file`	Ruta del archivo a importar
`--type`	Formato del archivo: json (por defecto), csv o tsv
`--jsonArray`	El archivo contiene un único array JSON, en vez de un documento JSON por línea
`--headerline`	(Solo CSV) La primera línea del archivo contiene los nombres de los campos
`--drop`	Elimina la colección de destino antes de importar (¡usar con cuidado!)

Atención. Por defecto, `mongoimport` espera un archivo con **un documento JSON por línea** (formato conocido como JSON Lines o NDJSON), no un único array JSON como los que hemos usado para ver ejemplos en este manual. Si tu archivo es un array JSON completo (empieza por `[` y termina por `]`), debes añadir la opción `--jsonArray`.

8.3 Importando datos en formato CSV

Cuando el archivo de origen es un CSV (por ejemplo, exportado desde una hoja de cálculo), necesitamos indicarle a `mongoimport` que interprete el formato correctamente:

```
mongoimport --db=tienda_online --collection=productos --type=csv --headerline --file=productos.csv
```

Con `--headerline`, `mongoimport` toma la primera fila del CSV como los nombres de los campos de cada documento resultante. Por ejemplo, un archivo CSV con esta estructura:

```
nombre,categoria,precio,stock
Teclado Mecánico,electronica,54.99,30
Silla de Oficina,hogar,129.99,12
```

generará documentos equivalentes a:

```
{ nombre: "Teclado Mecánico", categoria: "electronica", precio: "54.99", stock: "30" }
```

Idea clave. Fíjate en que, salvo que se indique lo contrario, `mongoimport` trata todos los valores de un CSV como **cadenas de texto** (incluso `precio` y `stock`, que a nosotros nos interesaría tener como números). Para que detecte automáticamente tipos numéricos y booleanos, se puede añadir la opción `--columnsHaveTypes` junto con una definición de tipos por columna, aunque para los catálogos de ejemplo de este curso normalmente nos bastará con revisar y corregir los tipos después de importar, con una `updateMany()` como las que vimos en el Capítulo 6.

8.4 mongoexport: extraer datos a un archivo

`mongoexport` hace el proceso inverso: toma los documentos de una colección (opcionalmente filtrados) y los escribe en un archivo JSON o CSV.

```
mongoexport --db=tienda_online --collection=productos --out=productos_export.json
```

También podemos exportar solo una parte de los documentos, usando un filtro de consulta en formato JSON (la misma sintaxis que usamos en `find()`). En este caso si utilizamos PowerShell tendremos que "escapar" las comillas dobles anteponiendo el símbolo \ :

```
mongoexport --db=tienda_online --collection=productos --query='{\"categoria\":
\"electronica\"}' --out=productos_electronica.json
```

Y para exportar a CSV, es obligatorio indicar explícitamente qué campos queremos incluir (CSV no tiene un concepto de documento anidado, así que MongoDB necesita saber qué "columnas" generar):

```
mongoexport --db=tienda_online --collection=productos --type=csv --
fields=nombre,categoria,precio,stock --out=productos.csv
```

Opción	Qué hace
`--out`	Ruta del archivo de salida
`--type`	Formato de salida: json (por defecto) o csv
`--query`	Filtro (igual sintaxis que find()) para exportar solo algunos documentos
`--fields`	(Obligatorio en CSV) Lista de campos a exportar, separados por comas
`--jsonArray`	Envuelve la salida JSON en un único array, en vez de un documento por línea

8.5 Un flujo de trabajo típico

Un caso de uso frecuente que puedes encontrarte en prácticas (y en un entorno profesional real) es: te entregan un archivo con datos de partida, lo importas a tu base de datos local para trabajar sobre

él con las operaciones CRUD que ya conoces, y al terminar exportas el resultado para entregarlo o para cargarlo en otro entorno.

- **Paso 1** — Recibimos `productos_iniciales.json` y lo importamos con `mongoimport --jsonArray` (porque nos lo entregan como un único array).
- **Paso 2** — Trabajamos sobre la colección con `find()`, `updateMany()`, etc., igual que en el Capítulo 6.
- **Paso 3** — Exportamos el resultado final con `mongoexport`, filtrando si es necesario con `--query`, para entregarlo o cargarlo en otro servidor.

Atención. Las herramientas `mongoimport`/`mongoexport` deben ejecutarse desde la terminal del sistema operativo, no desde dentro de mongosh. Si las escribes dentro del prompt de mongosh (por ejemplo, después de `test>`), obtendrás un error de sintaxis: primero hay que salir del shell con `exit` o abrir una terminal nueva.

Capítulo 9 · Copias de seguridad y restauración

mongodump y mongorestore: las herramientas estándar para proteger una base de datos MongoDB completa y poder recuperarla si algo sale mal.

9.1 Por qué no basta con mongoexport

En el Capítulo 8 vimos `mongoexport`, que genera archivos JSON o CSV legibles por humanos. Es una herramienta excelente para mover datos puntuales entre sistemas o compartir un subconjunto de una colección, pero tiene limitaciones importantes como estrategia de copia de seguridad:

- Exporta **una colección a la vez**: hacer una copia de seguridad completa de una base de datos con varias colecciones requeriría ejecutar el comando una vez por cada una.
- No conserva información como los **índices** definidos sobre la colección: al reimportar, habría que volver a crearlos manualmente.
- Convierte algunos tipos de datos BSON (como `ObjectId` o `Date`) a una representación de texto, lo que puede introducir ambigüedad al restaurar.

Para una copia de seguridad fiel y completa, MongoDB ofrece una pareja de herramientas distinta: **mongodump** y **mongorestore**, que trabajan con un formato binario (BSON) que preserva exactamente la estructura, los tipos de datos y los índices originales.

9.2 mongodump: crear una copia de seguridad

Sin ningún parámetro, `mongodump` se conecta al servidor local y vuelca **todas** las bases de datos y colecciones en una carpeta llamada `dump/` en el directorio actual:

```
mongodump
```

Lo habitual, sin embargo, es acotar la copia a lo que realmente necesitamos:

```
// Copia de seguridad de una sola base de datos
mongodump --db=tienda_online --out=./backups/2026-10-07

// Copia de seguridad de una sola colección dentro de esa base de datos
mongodump --db=tienda_online --collection=productos --out=./backups/2026-10-07
```

Opción	Qué hace
`--db`	Limita la copia a una base de datos concreta (si se omite, copia todas)
`--collection`	Limita la copia a una colección concreta (requiere --db)
`--out`	Carpeta de destino donde se guardará la copia

Opción	Qué hace
`--query`	Filtro (sintaxis de find()) para volcar solo algunos documentos de la colección
`--gzip`	Comprime los archivos de salida para ahorrar espacio

El resultado es una carpeta con un archivo `.bson` (los documentos) y un archivo `.metadata.json` (los índices y otras opciones de la colección) por cada colección copiada:

```
backups/2026-10-07/tienda_online/
  productos.bson
  productos.metadata.json
  pedidos.bson
  pedidos.metadata.json
```

Para saber más. Los archivos `.bson` generados por `mongodump` **no son legibles como texto** (a diferencia del JSON de `mongoexport`): son el formato binario interno de MongoDB. No intentes abrirlos con un editor de texto ni editarlos a mano; para inspeccionar su contenido legible, usa siempre `mongoexport` sobre la colección original, o restaura la copia y consúltala con `find()`.

9.3 mongorestore: recuperar una copia de seguridad

Para devolver los datos de una copia de seguridad a un servidor MongoDB (el mismo del que salieron, u otro distinto), usamos `mongorestore` apuntando a la carpeta generada por `mongodump`:

```
mongorestore --db=tienda_online ./backups/2026-10-07/tienda_online
```

Por defecto, `mongorestore` **no sobrescribe** colecciones que ya existan con los mismos nombres: añade los documentos de la copia a los que ya pudiera haber, lo que puede provocar documentos duplicados si los `_id` no coinciden exactamente con los originales. Si lo que queremos es una restauración limpia que sustituya por completo el contenido actual, debemos indicarlo explícitamente:

```
mongorestore --db=tienda_online --drop ./backups/2026-10-07/tienda_online
```

Atención. La opción `--drop` elimina cada colección de destino **antes** de restaurarla desde la copia. Es una operación destructiva sobre los datos actuales del servidor: antes de usarla en un entorno con datos reales, es imprescindible estar seguro de qué copia se está restaurando y sobre qué servidor.

9.4 Buenas prácticas de copias de seguridad

- **Automatiza y programa** las copias de seguridad (por ejemplo, con una tarea programada del sistema operativo que ejecute `mongodump` cada noche); una copia de seguridad que depende de que alguien se acuerde de ejecutarla a mano tiende a no existir cuando más se necesita.
- **Guarda las copias fuera del propio servidor** (otro disco, otra máquina, almacenamiento en la nube): una copia de seguridad guardada en el mismo disco que los datos originales no protege frente a un fallo de ese disco.
- **Verifica periódicamente que la restauración funciona**, no solo que la copia se generó sin errores. Una copia de seguridad que nunca se ha probado a restaurar es, en la práctica, una copia de seguridad en la que no se puede confiar del todo.
- **Conserva varias versiones** en el tiempo (por ejemplo, las copias de los últimos 7 días), no solo la más reciente: así es posible recuperar un estado anterior si el problema (un borrado accidental, una actualización incorrecta) no se detecta de inmediato.

Idea clave. MongoDB Atlas (la plataforma en la nube de MongoDB, que veremos en la Unidad de Trabajo 2) incluye copias de seguridad automáticas gestionadas, con restauración a un punto concreto en el tiempo, sin necesidad de ejecutar `mongodump`/`mongorestore` manualmente. En este manual nos centramos en las herramientas de línea de comandos porque funcionan igual en cualquier instalación local de MongoDB, incluida la que usamos en el aula.

Capítulo 10 · Actividad 2: práctica integral de CRUD

El enunciado completo de la Actividad 2, ya realizada en el aula, con la solución de referencia de cada ejercicio. Úsalo para repasar, comparar tus propias sentencias y entender el porqué de cada corrección recibida.

10.1 Planteamiento de la actividad

Esta actividad se trabaja sobre la colección `productos` de la base de datos `tienda_online`, el mismo catálogo de ejemplo que hemos usado a lo largo de todo este manual. Consta de 14 ejercicios organizados según las cuatro operaciones CRUD (Create, Read, Update, Delete), más dos ejercicios adicionales de tipo "extra" que anticipan el framework de agregación que estudiaremos en profundidad en la Unidad de Trabajo 2.

Para saber más. Este capítulo recoge la actividad **tal y como ya se ha realizado en clase**, junto con la solución de referencia de cada ejercicio. No es un ejercicio nuevo: su objetivo es servir de material de repaso y autocorrección, especialmente a la vista de los errores más habituales detectados durante la corrección, que se señalan en los avisos de cada apartado.

10.2 CREATE

Ejercicio 1. Inserta un producto que no tenga stock con estas especificaciones:

```
"nombre": "Auriculares Inalámbricos",
"categoria": "electronica",
"precio": 129.99,
"stock": 0,
"marca": "SoundWave",
"caracteristicas": ["Cancelación ruido", "20h batería", "Carga rápida"]
```

Solución de referencia:

```
db.productos.insertOne({
  nombre: "Auriculares Inalámbricos",
  categoria: "electronica",
  precio: 129.99,
  stock: 0,
  marca: "SoundWave",
  caracteristicas: ["Cancelación ruido", "20h batería", "Carga rápida"]
})
```

Idea clave. Un "producto que no tenga stock" significa `stock: 0`, no la ausencia del campo `stock`. Omitir el campo por completo también sería técnicamente válido gracias al esquema flexible de

MongoDB (Capítulo 5), pero dificulta luego consultarlo de forma consistente junto con el resto de productos, que sí tienen ese campo.

10.3 READ

Ejercicio 2. Muestra el nombre, precio y marca de todos los productos de electrónica.

```
db.productos.find(
  { categoria: "electronica" },
  { nombre: 1, precio: 1, marca: 1, _id: 0 }
)
```

Ejercicio 3. Busca todos los productos con precio mayor que 100€ y que tengan al menos 30 de stock.

```
db.productos.find({
  precio: { $gt: 100 },
  stock: { $gte: 30 }
})
```

Atención. "Al menos 30" significa 30 incluido, por lo que el operador correcto es `$gte` (mayor o igual que), no `$gt`. Confundir `$gt`/`$gte` o `$lt`/`$lte` es uno de los descuidos más fáciles de cometer —y más fáciles de evitar releyendo el enunciado antes de escribir la consulta—.

Ejercicio 4. Muestra todos los productos de las marcas SportMax, HomeChef y PhoneTech en una única sentencia.

```
db.productos.find({
  marca: { $in: ["SportMax", "HomeChef", "PhoneTech"] }
})
```

Atención. Los valores de texto en MongoDB distinguen mayúsculas de minúsculas: `"SportMax"` y `"sportmax"` son, para una consulta exacta como `$in`, dos cadenas completamente distintas. Si al revisar los resultados de este ejercicio te faltan productos que deberían aparecer, el primer sospechoso es casi siempre una diferencia de mayúsculas/minúsculas entre el valor consultado y el valor realmente almacenado.

Ejercicio 5. Muestra los 5 productos que más stock tengan en la base de datos.

```
db.productos.find().sort({ stock: -1 }).limit(5)
```

Idea clave. `sort()` sin `limit()` ordena toda la colección pero la devuelve entera; `limit()` sin `sort()` devuelve "los 5 primeros que encuentre", en un orden no garantizado. Para un enunciado como este ("los 5 que más...") ambos métodos son imprescindibles y en este orden: primero ordenar, después limitar.

10.4 UPDATE

Ejercicio 6. Actualiza el precio del Smartphone Pro a 649.99.

```
db.productos.updateOne(
  { nombre: "Smartphone Pro" },
  { $set: { precio: 649.99 } }
)
```

Ejercicio 7. Aumenta en 10 el stock de todos los productos de deportes.

```
db.productos.updateMany(
  { categoria: "deportes" },
  { $inc: { stock: 10 } }
)
```

Ejercicio 8. Añade la característica "Garantía 2 años" a los productos de la marca "TabTech".

```
db.productos.updateMany(
  { marca: "TabTech" },
  { $push: { caracteristicas: "Garantía 2 años" } }
)
```

Ejercicio 9. Añade un campo al producto Laptop Elite que contenga una fecha de la última actualización.

```
db.productos.updateOne(
  { nombre: "Laptop Elite" },
  { $set: { fecha_actualizacion: new Date() } }
)
```

Atención. Los ejercicios 7 a 9 son el bloque donde más errores aparecen al corregir, y casi siempre por el mismo motivo: `updateOne()`/`updateMany()` necesitan **dos argumentos separados** —el filtro (qué documentos afectar) y las modificaciones (qué cambiar)—. Usar `updateOne()` donde el enunciado

pide "todos los productos de..." (que debería ser `updateMany()`) es un error de lectura del enunciado que no da ningún error en mongosh: la sentencia se ejecuta sin problema, pero `matchedCount` revela que solo se ha modificado un documento en lugar de todos los que cumplían la condición. Conviene revisar siempre el resultado (`matchedCount`, `modifiedCount`) antes de dar un ejercicio de update por terminado, igual que en el Ejercicio 9 conviene usar `new Date()` —un valor de tipo Date real (Capítulo 4)— y no un número o una cadena de texto con apariencia de fecha.

10.5 DELETE

Ejercicio 10. Elimina el producto Tablet Vision de la marca TabTech.

```
db.productos.deleteOne({
  nombre: "Tablet Vision",
  marca: "TabTech"
})
```

Ejercicio 11. Elimina todos los productos sin stock.

```
db.productos.deleteMany({ stock: 0 })
```

Ejercicio 12. Elimina los productos de ropa que tengan un valor inferior a 20€.

```
db.productos.deleteMany({
  categoria: "ropa",
  precio: { $lt: 20 }
})
```

Atención. El error más grave detectado en este bloque durante la corrección fue escribir el filtro como **segundo** argumento en lugar de como único argumento de `deleteOne()`/`deleteMany()` —por ejemplo, `db.productos.deleteMany({}, { categoria: "ropa", precio: { $lt: 20 } })`—. A diferencia de `updateOne()`/`updateMany()`, que sí reciben dos argumentos (filtro y cambios), `deleteOne()`/`deleteMany()` reciben **un único argumento**: el filtro. Escribirlo como segundo argumento hace que MongoDB lo ignore por completo y elimine documentos siguiendo el primer argumento (en el ejemplo, un filtro vacío `{}` que no excluye nada), lo cual es **irreversible** y en algún caso real de corrección llegó a borrar varios productos que no debían eliminarse. Por eso insistimos tanto, desde el Capítulo 6, en la norma de oro: antes de ejecutar cualquier `deleteOne()`/`deleteMany()`, ejecuta el mismo filtro con `find()` y comprueba que los documentos que aparecen son exactamente los que quieres borrar.

10.6 EXTRA (agregación)

Ejercicio 13. Cuenta los productos que hay en cada categoría.

```
db.productos.aggregate([
  { $group: { _id: "$categoria", total: { $sum: 1 } } }
])
```

Idea clave. Al revisar este ejercicio en las colecciones reales de clase, es habitual encontrar categorías "ruido" en el resultado —variantes con tilde, mayúsculas sueltas, o valores que no deberían existir como categoría propia—, fruto de inserciones manuales hechas durante las prácticas de ejercicios anteriores. Es un buen ejercicio práctico en sí mismo: antes de dar por buena esta agregación, conviene revisar la lista de categorías resultante y plantearse si todas son "reales" o si alguna es, en realidad, un error de los ejercicios de Create/Update anteriores.

Ejercicio 14. Calcula el precio promedio de cada marca.

```
db.productos.aggregate([
  { $group: { _id: "$marca", precioPromedio: { $avg: "$precio" } } }
])
```

Atención. El error de sintaxis más habitual en este ejercicio es olvidar el símbolo `$` delante del nombre del campo dentro de la etapa `$group` (escribir `marca` en vez de `$marca`, o `precio` en vez de `$precio`). Sin el `$`, MongoDB no lo interpreta como "el valor de ese campo en cada documento", sino como una cadena de texto literal, y la agregación falla o produce un resultado sin sentido. Recuerda también que `_id` es el nombre obligatorio del campo de agrupación dentro de `$group` (Capítulo 7); usar cualquier otro nombre ahí —como `id`, sin guion bajo— no es válido.

10.7 Para repasar

Si comparas tus propias respuestas con las de este capítulo y encuentras diferencias, no te limites a copiar la solución: vuelve a ejecutar tu versión original en mongosh, observa qué `matchedCount`, `modifiedCount` o `deletedCount` produce realmente, y contrástalo con lo que el enunciado pedía. Entender **por qué** una sentencia no hacía lo que parecía que iba a hacer es, con diferencia, el aprendizaje más valioso de esta actividad, y es exactamente el mismo proceso de verificación que deberás aplicar en los ejercicios y actividades de evaluación de la Unidad de Trabajo 2.

Capítulo 11 · Cierre de la unidad y próximos pasos

Un repaso de todo lo recorrido en esta Unidad de Trabajo y una mirada hacia lo que viene en la Unidad de Trabajo 2.

11.1 Lo que hemos aprendido

A lo largo de esta unidad hemos construido, paso a paso, una base sólida sobre bases de datos NoSQL y MongoDB, partiendo de los conceptos más generales hasta llegar a dominar las operaciones del día a día de cualquier desarrollador que trabaje con esta tecnología:

- **Qué son las bases de datos NoSQL** y por qué surgieron como alternativa (no como sustituto) al modelo relacional, junto con sus principales familias: documentales, clave-valor, columnares y de grafos.
- **Qué es MongoDB** en concreto: su arquitectura basada en documentos, colecciones y bases de datos, y su ecosistema de herramientas (servidor, mongosh, Compass, y una introducción a Atlas).
- **Instalación y primeros pasos**: cómo poner en marcha un servidor MongoDB local y conectarnos a él con mongosh.
- **Tipos de datos BSON**: las diferencias entre JSON y BSON, el papel central del campo `_id` y el tipo `ObjectId`, y tipos especiales como `Date` o `Decimal128`.
- **Bases de datos y colecciones**: su creación perezosa o explícita, su administración, y la flexibilidad (y disciplina necesaria) del esquema de MongoDB.
- **Las operaciones CRUD completas**: crear documentos con `insertOne()`/`insertMany()`, consultarlos con `find()` y una amplia variedad de operadores de comparación y lógicos, actualizarlos con `updateOne()`/`updateMany()` y los operadores de actualización, y eliminarlos con `deleteOne()`/`deleteMany()`.
- **Índices**: su fundamento teórico (árboles B-Tree), los distintos tipos disponibles, y cómo diseñarlos correctamente (regla ESR) y verificar su efecto real con `explain()`.
- **Importación, exportación, copias de seguridad y restauración**: cómo mover datos dentro y fuera de MongoDB de forma segura con `mongoimport`/`mongoexport` y `mongodump`/`mongorestore`.

Y, de forma transversal a todo lo anterior, hemos practicado y consolidado estos conceptos a través de la Actividad 2 de CRUD, revisada en detalle en el Capítulo 10, cuyos errores más habituales (el filtro como argumento equivocado, la diferencia entre `updateOne()` y `updateMany()`, la sensibilidad a mayúsculas/minúsculas en los valores de texto) son, en realidad, los mismos matices que distinguen a quien conoce MongoDB de memoria de quien realmente entiende cómo razona esta base de datos.

11.2 Autoevaluación: ¿dominas ya estos conceptos?

Antes de avanzar a la siguiente unidad, es un buen momento para comprobar, con sinceridad, si podrías responder sin dificultad a preguntas como estas. Si alguna te genera dudas, es el momento de volver al capítulo correspondiente y repasarlo, no de avanzar "a medias":

- ¿Sabrías explicar con tus propias palabras la diferencia entre una base de datos relacional y una documental, y dar un ejemplo de cuándo elegirías cada una?
- ¿Entiendes por qué `db.productos.find()` se lee como "colección productos, operación find", y qué representa cada una de esas tres partes?
- ¿Sabrías escribir, sin consultar el manual, una consulta con al menos dos condiciones combinadas con `$and` o `$or`, con proyección y con `sort()`+`limit()`?
- ¿Podrías explicar por qué `updateOne()` y `deleteOne()` reciben un número distinto de argumentos, y qué consecuencias tiene confundirlos?
- ¿Sabrías diseñar un índice compuesto aplicando la regla ESR para una consulta concreta, y comprobar con `explain()` si realmente se está usando?

11.3 Qué viene después: Unidad de Trabajo 2

La Unidad de Trabajo 2, "MongoDB Avanzado y Atlas", construye directamente sobre todo lo trabajado aquí. Allí profundizaremos en aspectos que en esta unidad solo hemos mencionado de pasada como anticipo, entre ellos:

- **MongoDB Atlas**, la plataforma en la nube, incluyendo sus copias de seguridad gestionadas y herramientas de visualización con Charts.
- **El framework de agregación**: el pipeline de etapas (`$match`, `$group`, `$project`, `$lookup`, `$facet`...) para construir consultas analíticas que van mucho más allá de lo que permite `find()`.
- **Modelado de datos avanzado**: cuándo embeber documentos y cuándo referenciarlos entre colecciones, y las implicaciones de cada decisión.
- **Validación de esquemas con `$jsonSchema`**, para exigir estructura y tipos concretos incluso en una base de datos NoSQL.